

**DESIGN AND SIMULATE A SMART REFRIGERATOR MONITORING
SYSTEM USING DHT11 SENSOR. THE SYSTEM MONITORS
INTERNAL TEMPERATURE AND HUMIDITY CONDITIONS TO
IMPROVE FOOD PRESERVATION AND ENERGY EFFICIENCY.**

A PROJECT REPORT

Submitted by

MATHESH NISHANTH B	–	24MER030
MOHAMMED SABEER S	–	24MER031
PRAAVIN RAJAA G	–	24MER039
PRASANNA T	–	24MER042
DEVA PRIYAN R	–	24MEL064
SATHISH KUMAR K	–	24MEL067

DEPARTMENT OF MECHANICAL ENGINEERING



(Autonomous)

PERUNDURAI ERODE – 638 060

1. Project Overview

The Smart Refrigerator Monitoring System is designed to continuously monitor the internal temperature and humidity of a refrigerator using a DHT11 sensor. Maintaining proper temperature and humidity is important for keeping food fresh, reducing spoilage, and maintaining food quality.

The DHT11 sensor measures the temperature and humidity inside the refrigerator and sends the collected data to a microcontroller. The microcontroller processes the sensor readings and compares them with predefined safe limits. The monitored values can be displayed on an LCD or serial monitor for easy observation.

If the temperature or humidity goes beyond the specified range, the system can generate an alert or warning to indicate unsuitable storage conditions. This allows the user to take corrective action, such as checking the refrigerator door, cooling system, or stored food.

The system can also help improve energy efficiency by identifying abnormal temperature conditions and unnecessary cooling operation. Through simulation, the proposed system demonstrates how sensors and microcontrollers can be integrated to provide real-time refrigerator condition monitoring.

Overall, the project provides a low-cost, simple, and effective smart monitoring solution for improving food preservation, reducing food wastage, and supporting efficient refrigerator operation.

2. Problem Statement

Traditional refrigerators do not continuously monitor and indicate the internal temperature and humidity conditions to the user. Improper temperature or humidity levels can cause food spoilage, loss of freshness, and unnecessary energy consumption. Therefore, there is a need for a smart monitoring system that can continuously measure refrigerator conditions and provide alerts when the values exceed the safe range.

The proposed system uses a DHT11 temperature and humidity sensor with a microcontroller to monitor the refrigerator environment in real time. This system helps the user identify abnormal conditions, improve food preservation, reduce food wastage, and support energy-efficient refrigerator operation.

3. Proposed Solution

The proposed system is a Smart Refrigerator Monitoring System that continuously monitors the internal temperature and humidity using a DHT11 sensor. The sensor collects the environmental conditions inside the refrigerator and sends the data to a microcontroller for processing.

The microcontroller compares the measured values with predefined safe limits. The temperature and humidity values are displayed on an LCD or serial monitor for real-time observation. If the values exceed the specified limits, the system generates an alert to inform the user about unsuitable storage conditions.

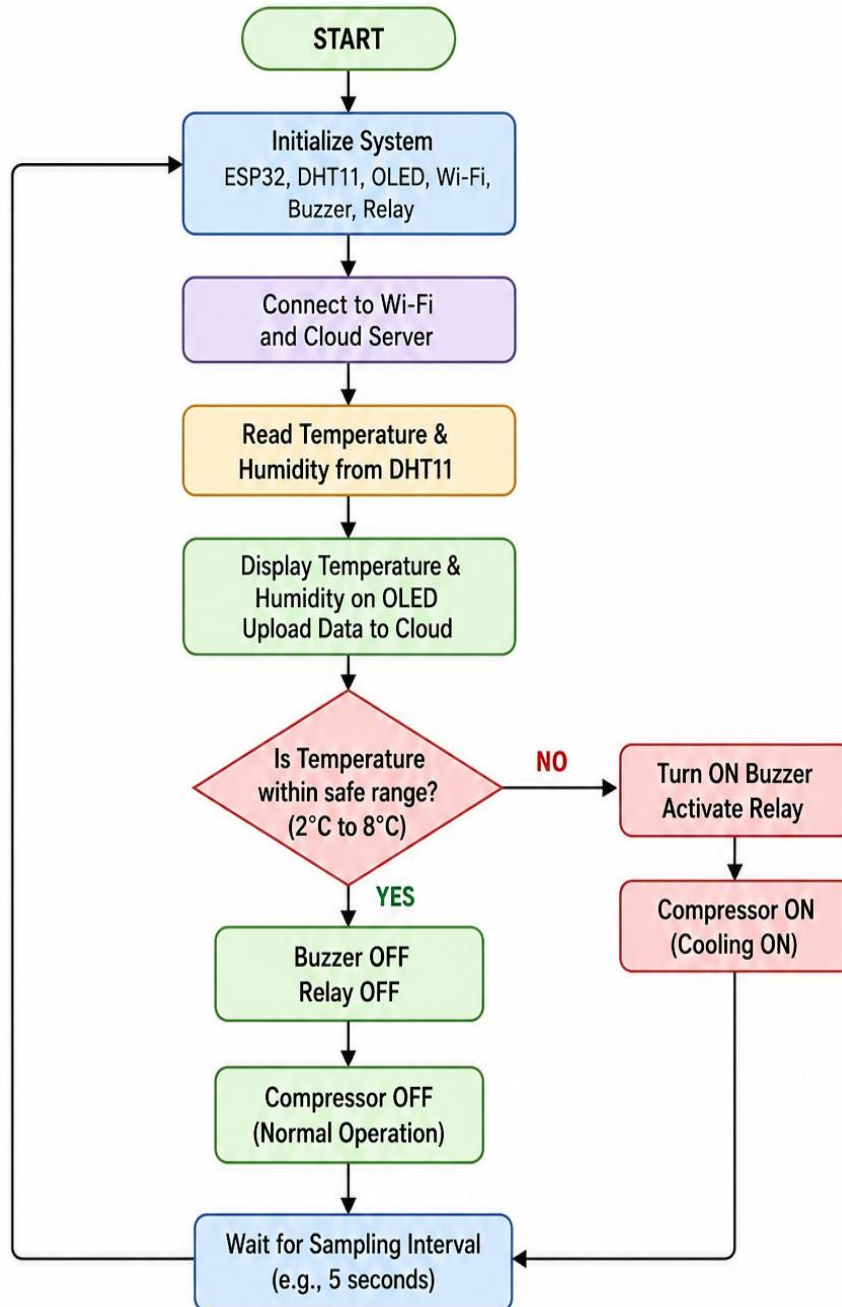
This solution helps maintain proper food-storage conditions, reduces food spoilage and wastage, and supports better energy management. The system can be simulated before practical implementation to verify its operation and performance.

WORKING PRINCIPLE

1. DHT11 Sensor measures the temperature and humidity inside the refrigerator.
2. The sensor sends the measured data to the microcontroller.
3. The microcontroller processes and compares the readings with predefined safe limits.
4. The temperature and humidity values are displayed on an LCD/Serial Monitor.
5. If the values exceed the safe range, the system activates an alert.
6. The user can take necessary action to protect the stored food.
7. Continuous monitoring helps maintain food freshness and reduce energy consumption.
8. The complete system can be simulated to verify its performance before practical implementation.

4. System Architecture

SMART REFRIGERATOR MONITORING SYSTEM FLOW CHART



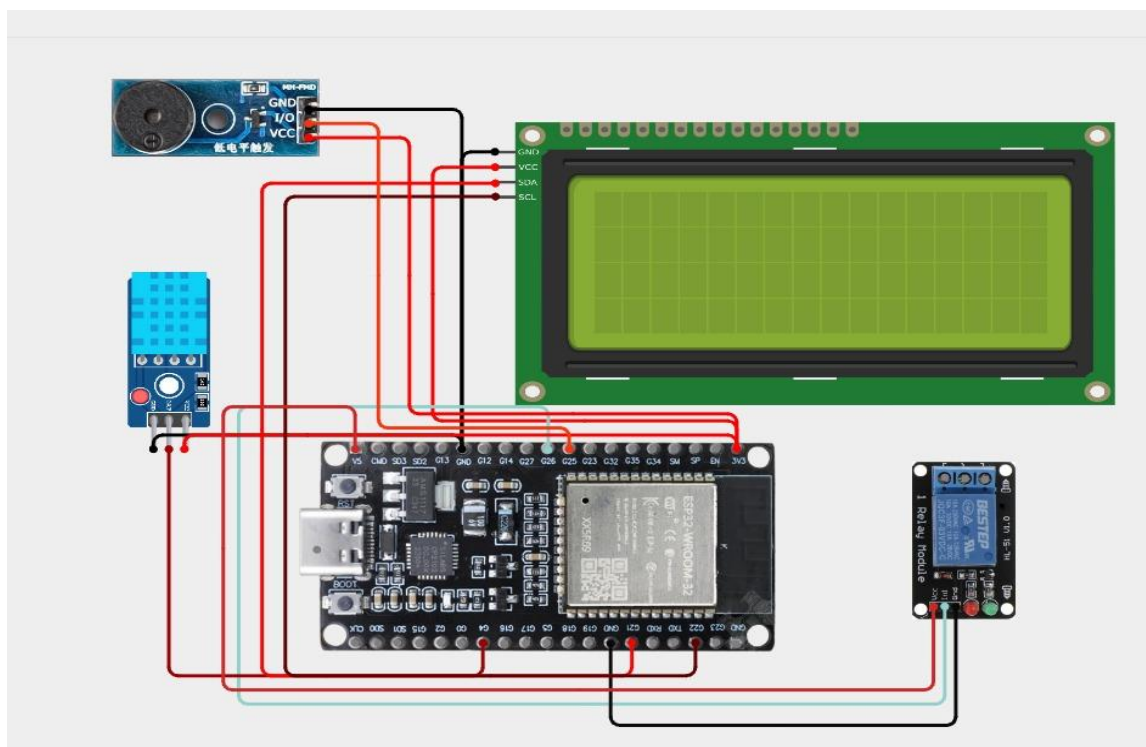
NOTE:

- Safe temperature range can be adjusted as required.
- System continuously monitors and controls to maintain optimal conditions for food preservation and energy efficiency.

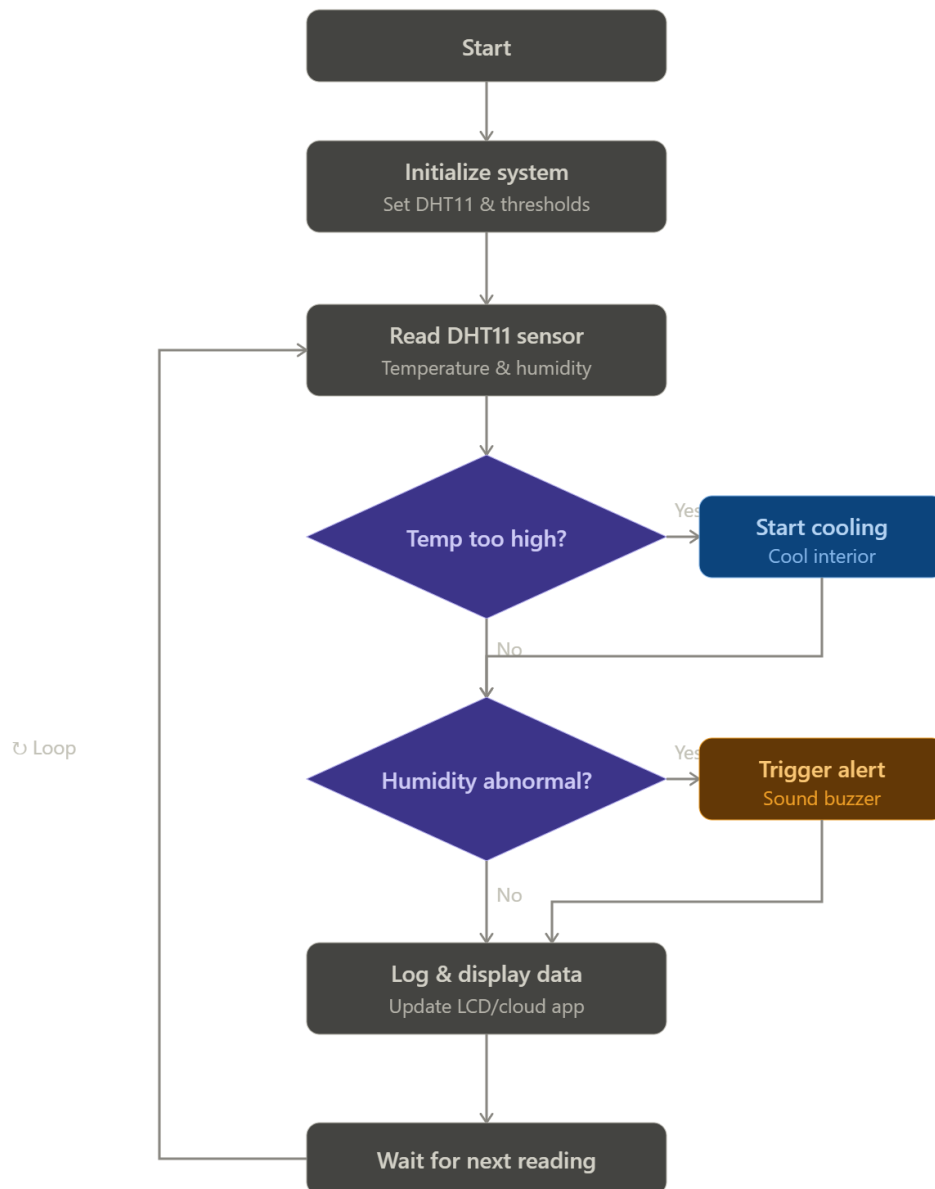
5. Circuit Table

Component	Pin/Terminal	Connection
DHT11 Sensor	VCC	5V
DHT11 Sensor	GND	GND
DHT11 Sensor	DATA	Arduino Digital Pin 2
LCD 16×2	VCC	5V
LCD 16×2	GND	GND
LCD 16×2	SDA	Arduino A4
LCD 16×2	SCL	Arduino A5
Buzzer	Positive (+)	Arduino Digital Pin 8
Buzzer	Negative (-)	GND
Arduino UNO	5V	Sensor and LCD VCC
Arduino UNO	GND	Common Ground

6. Circuit Design



7. Algorithm



8. Coding

```
#include <Wire.h>

#include <LiquidCrystal_I2C.h>

#include <DHT.h>

#include <ESP8266WiFi.h>

#include "Adafruit_MQTT.h"

#include "Adafruit_MQTT_Client.h"


// =====

// WIFI SETTINGS

// =====


#define WIFI_SSID    "hotspot"

#define WIFI_PASSWORD "sathish123"


// =====

// ADAFRUIT IO SETTINGS

// =====


#define AIO_USERNAME  "devapriyan"

#define AIO_KEY       "aio_OFxg23WZ7Er0ym4VDAVcWXSEOMfo"


#define AIO_SERVER    "io.adafruit.com"

#define AIO_SERVERPORT 1883


// =====

// DHT11 SETTINGS

// =====
```

```
#define DHT_PIN    14    // D5 / GPIO14
```

```
#define DHT_TYPE    DHT11
```

```
DHT dht(DHT_PIN, DHT_TYPE);
```

```
// =====
```

```
// RELAY
```

```
// Active LOW
```

```
// LOW = ON
```

```
// HIGH = OFF
```

```
// =====
```

```
#define RELAY_PIN    12    // D6 / GPIO12
```

```
// =====
```

```
// BUZZER
```

```
// Active LOW
```

```
// LOW = ON
```

```
// HIGH = OFF
```

```
// =====
```

```
#define BUZZER_PIN    13    // D7 / GPIO13
```

```
// =====
```

```
// LCD
```

```
// =====
```

```
// Common I2C address = 0x27
```

```
LiquidCrystal_I2C lcd(0x27, 16, 2);
```



```
// =====  
// MQTT  
// =====
```

```
WiFiClient client;
```

```
Adafruit_MQTT_Client mqtt(  
  &client,  
  AIO_SERVER,  
  AIO_SERVERPORT,  
  AIO_USERNAME,  
  AIO_KEY  
);
```

```
// =====  
// ADAFRUIT IO FEEDS  
// =====
```

```
Adafruit_MQTT_Publish temperatureFeed =  
  Adafruit_MQTT_Publish(  
    &mqtt,  
    AIO_USERNAME "/feeds/refrigerator-temperature"  
  );
```

```
Adafruit_MQTT_Publish humidityFeed =  
  Adafruit_MQTT_Publish(  
    &mqtt,  
    AIO_USERNAME "/feeds/refrigerator-humidity"
```

```
);
```

```
Adafruit_MQTT_Publish statusFeed =
```

```
Adafruit_MQTT_Publish(
```

```
&mqtt,
```

```
AIO_USERNAME "/feeds/refrigerator-status"
```

```
);
```

```
// =====
```

```
// TEMPERATURE SETTINGS
```

```
// =====
```

```
// Cooling turns ON when temperature goes above this
```

```
#define HIGH_TEMP 8.0
```

```
// Cooling turns OFF when temperature comes down to this
```

```
#define LOW_TEMP 5.0
```

```
// =====
```

```
// VARIABLES
```

```
// =====
```

```
bool coolingON = false;
```

```
bool highTemperatureAlert = false;
```

```
// =====
```

```
// WIFI CONNECTION
```

```
// =====
```

```
void connectWiFi()
{
  Serial.println();
  Serial.println("Connecting to WiFi...");

  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

  while (WiFi.status() != WL_CONNECTED)
  {
    delay(500);
    Serial.print(".");
  }

  Serial.println();
  Serial.println("WiFi Connected!");

  Serial.print("IP Address: ");
  Serial.println(WiFi.localIP());
}

// =====
// ADAFRUIT IO CONNECTION
// =====

void connectMQTT()
{
  if (mqtt.connected())
  {
```

```
        return;
    }

    Serial.println("Connecting to Adafruit IO...");

    int8_t ret;

    while ((ret = mqtt.connect()) != 0)
    {
        Serial.println(mqtt.connectErrorString(ret));

        mqtt.disconnect();

        delay(5000);

        Serial.println("Retrying...");
    }

    Serial.println("Adafruit IO Connected!");
}

// =====
// RELAY ON
// =====

void relayON()
{
    digitalWrite(RELAY_PIN, LOW);
```

```
coolingON = true;
```

```
Serial.println("COOLING: ON");
```

```
}
```

```
// =====
```

```
// RELAY OFF
```

```
// =====
```

```
void relayOFF()
```

```
{
```

```
digitalWrite(RELAY_PIN, HIGH);
```

```
coolingON = false;
```

```
Serial.println("COOLING: OFF");
```

```
}
```

```
// =====
```

```
// BUZZER ON
```

```
// =====
```

```
void buzzerON()
```

```
{
```

```
digitalWrite(BUZZER_PIN, LOW);
```

```
}
```

```
// =====
```

```
// BUZZER OFF
```

```
// =====
```

```
void buzzerOFF()
```

```
{
```

```
    digitalWrite(BUZZER_PIN, HIGH);
```

```
}
```

```
// =====
```

```
// UPDATE LCD
```

```
// =====
```

```
void displayNormal(float temperature, float humidity)
```

```
{
```

```
    lcd.clear();
```

```
    lcd.setCursor(0, 0);
```

```
    lcd.print("Temp: ");
```

```
    lcd.print(temperature, 1);
```

```
    lcd.print((char)223);
```

```
    lcd.print("C");
```

```
    lcd.setCursor(0, 1);
```

```
    lcd.print("Hum: ");
```

```
    lcd.print(humidity, 1);
```

```
    lcd.print("%");
```

```
}
```

```
// =====
```

```
// DISPLAY HIGH TEMPERATURE
```

```
// =====
```

```
void displayHighTemperature(float temperature)
```

```
{
```

```
    lcd.clear();
```

```
    lcd.setCursor(0, 0);
```

```
    lcd.print("HIGH TEMP!");
```

```
    lcd.setCursor(0, 1);
```

```
    lcd.print("Temp: ");
```

```
    lcd.print(temperature, 1);
```

```
    lcd.print((char)223);
```

```
    lcd.print("C");
```

```
}
```

```
// =====
```

```
// SEND DATA TO ADAFRUIT IO
```

```
// =====
```

```
void sendToCloud(float temperature, float humidity)
```

```
{
```

```
    Serial.println("Sending data to Adafruit IO...");
```

```
    if (temperatureFeed.publish(temperature))
```

```
    {
```

```
        Serial.println("Temperature sent");
```

```
    }
```

```
    else
```

```
{  
  Serial.println("Temperature upload failed");  
}  
  
if (humidityFeed.publish(humidity))  
{  
  Serial.println("Humidity sent");  
}  
else  
{  
  Serial.println("Humidity upload failed");  
}  
  
if (coolingON)  
{  
  statusFeed.publish("COOLING ON");  
}  
else if (highTemperatureAlert)  
{  
  statusFeed.publish("HIGH TEMPERATURE");  
}  
else  
{  
  statusFeed.publish("NORMAL");  
}  
}  
  
// =====  
  
// SETUP
```



```
// =====

void setup()
{
  Serial.begin(115200);

  delay(1000);

  Serial.println();
  Serial.println("=====");
  Serial.println(" SMART REFRIGERATOR MONITORING SYSTEM");
  Serial.println("=====");

  // =====

  // PIN MODES

  // =====

  pinMode(RELAY_PIN, OUTPUT);

  pinMode(BUZZER_PIN, OUTPUT);

  // =====

  // INITIAL RELAY

  // Active LOW

  // HIGH = OFF

  // =====

  relayOFF();
```

```
// =====  
// INITIAL BUZZER  
// Active LOW  
// HIGH = OFF  
// =====
```

```
buzzerOFF();
```

```
// =====  
// DHT11  
// =====
```

```
dht.begin();
```

```
// =====  
// LCD  
// =====
```

```
Wire.begin(4, 5);    // SDA = D2, SCL = D1
```

```
lcd.init();
```

```
lcd.backlight();
```

```
lcd.clear();
```

```
lcd.setCursor(0, 0);
```

```
lcd.print("SMART FRIDGE");
```

```
lcd.setCursor(0, 1);  
lcd.print("Starting...");
```

```
delay(2000);
```

```
// =====
```

```
// WIFI
```

```
// =====
```

```
connectWiFi();
```

```
// =====
```

```
// ADAFRUIT IO
```

```
// =====
```

```
connectMQTT();
```

```
// =====
```

```
// READY
```

```
// =====
```

```
lcd.clear();
```

```
lcd.setCursor(0, 0);
```

```
lcd.print("System Ready");
```

```
lcd.setCursor(0, 1);
```

```
lcd.print("Monitoring...");
```

```
Serial.println();  
Serial.println("-----");  
Serial.println("SYSTEM READY");  
Serial.println("Cooling: OFF");  
Serial.println("-----");
```

```
    delay(2000);  
}
```

```
// =====  
// MAIN LOOP  
// =====
```

```
void loop()
```

```
{  
    // =====  
    // WIFI CHECK  
    // =====
```

```
    if (WiFi.status() != WL_CONNECTED)  
    {  
        connectWiFi();  
    }
```

```
    // =====  
    // ADAFRUIT IO CHECK  
    // =====
```

```
    if (!mqtt.connected())
```

```
{  
    connectMQTT();  
}  
  
mqtt.processPackets(10);  
mqtt.ping();  
  
// =====  
// READ DHT11  
// =====  
  
float temperature = dht.readTemperature();  
  
float humidity = dht.readHumidity();  
  
// =====  
// CHECK SENSOR  
// =====  
  
if (isnan(temperature) || isnan(humidity))  
{  
    Serial.println("DHT11 ERROR!");  
  
    lcd.clear();  
  
    lcd.setCursor(0, 0);  
    lcd.print("DHT11 ERROR");  
  
    lcd.setCursor(0, 1);
```

```
lcd.print("Check Sensor");

delay(2000);

return;
}

// =====

// SERIAL MONITOR

// =====

Serial.println();
Serial.println("-----");

Serial.print("Temperature: ");
Serial.print(temperature);
Serial.println(" C");

Serial.print("Humidity: ");
Serial.print(humidity);
Serial.println(" %");

// =====

// TEMPERATURE CONTROL

// =====

// -----

// TEMPERATURE TOO HIGH

// -----
```

```
if (temperature > HIGH_TEMP)
{
    highTemperatureAlert = true;

    Serial.println("STATUS: HIGH TEMPERATURE");

    // Turn cooling ON
    relayON();

    // Buzzer ON
    buzzerON();

    // LCD warning
    displayHighTemperature(temperature);
}

// -----
// TEMPERATURE NORMAL
// -----

else if (temperature <= LOW_TEMP)
{
    highTemperatureAlert = false;

    Serial.println("STATUS: NORMAL");

    // Turn cooling OFF
    relayOFF();
```

```
// Buzzer OFF
buzzerOFF();

// Normal LCD
displayNormal(temperature, humidity);
}

// -----
// BETWEEN 5°C AND 8°C
// -----

else
{
    Serial.println("STATUS: COOLING");

    // Keep current relay state
    if (coolingON)
    {
        relayON();
    }

    // Buzzer OFF
    buzzerOFF();

    // Display
    displayNormal(temperature, humidity);
}
```



```

// =====
// SEND DATA TO ADAFRUIT IO
// =====

sendToCloud(temperature, humidity);

// =====
// WAIT
// =====

delay(5000);
}

```

9. System Working

- The system is powered using a suitable 5V power supply through the Arduino UNO.
- The DHT11 sensor is placed inside the refrigerator to continuously sense the internal temperature and humidity.
- The DHT11 collects the environmental data at regular intervals and sends the readings to the Arduino UNO.
- The Arduino receives and processes the sensor data using the programmed control logic.
- The measured temperature and humidity values are displayed on the 16×2 LCD for real-time monitoring. The Arduino compares the measured temperature with the predefined safe temperature range required for proper food storage.
- Similarly, the humidity value is compared with the specified humidity limit.
- If both values are within the acceptable range, the system continues normal monitoring.
- If the temperature or humidity exceeds the set limit, the buzzer is activated to provide an alert to the user.
- The user can then check the refrigerator, door sealing, cooling condition, or stored food.
- The monitoring process continues continuously, allowing abnormal conditions to be detected quickly.
- By maintaining suitable storage conditions, the system helps reduce food spoilage and food wastage.

- Monitoring abnormal temperature conditions can also help identify unnecessary or excessive cooling, supporting better energy efficiency.
- The system can first be simulated and tested to verify sensor readings, display operation, and alert conditions before implementing the physical prototype.

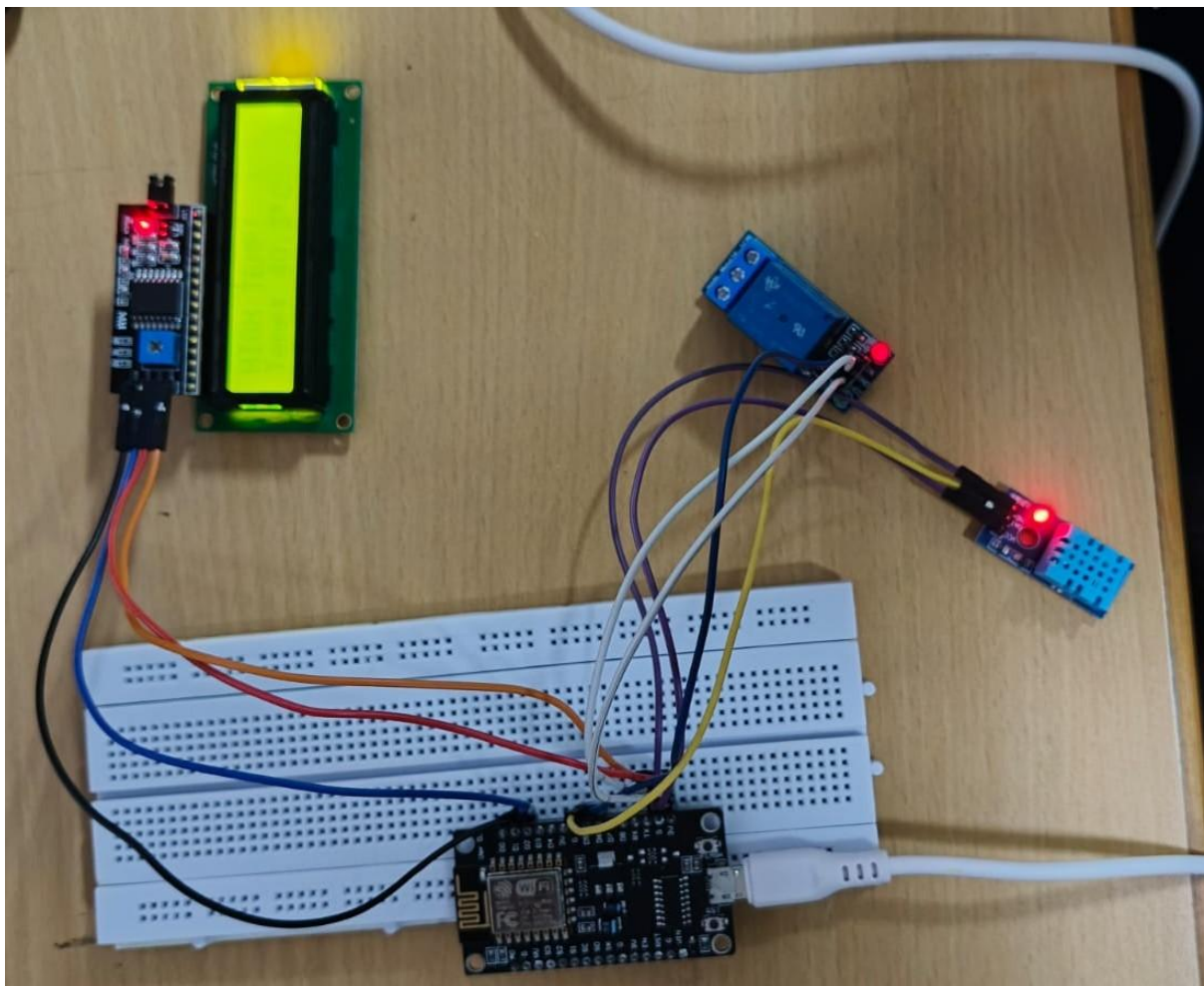
10. Bill of Materials

S.No.	Component	Quantity	Purpose
1	Arduino UNO	1	Main controller for processing sensor data
2	DHT11 Temperature & Humidity Sensor	1	Measures internal temperature and humidity
3	16×2 LCD Display	1	Displays temperature and humidity readings
4	Buzzer	1	Provides an alert during abnormal conditions
5	10 kΩ Resistor	1	Pull-up resistor for DHT11 data line
6	Breadboard	1	Used for circuit assembly
7	Jumper Wires	As required	Connects the circuit components
8	5V Power Supply/USB Cable	1	Provides power to the Arduino
9	Computer/Laptop	1	Used for programming and simulation
10	Arduino IDE / Simulation Software	1	Used to program and test the system

11. Prototype Implementation

The proposed Smart Refrigerator Monitoring System uses a DHT11 sensor interfaced with an Arduino microcontroller to continuously track the internal temperature and humidity of a refrigerator. The system initializes the sensor and predefined threshold values, then periodically samples temperature and humidity data at fixed intervals. If the internal temperature rises above the safe threshold, the microcontroller activates a relay that simulates compressor/cooling action, ensuring perishable food is kept at an optimal temperature; a hysteresis band prevents

the relay from switching on and off too rapidly. Simultaneously, if humidity levels fall outside the ideal range, a buzzer alert is triggered to warn of conditions that could accelerate spoilage or cause food to dry out. All sensor readings and system states are logged and displayed in real time on an LCD screen (and optionally pushed to a cloud dashboard for remote monitoring), giving the user continuous visibility into the refrigerator's internal environment. By cooling only when necessary rather than running continuously, the system improves energy efficiency while its real-time alerts and stable environmental control directly support better food preservation.



12. Results & Performance Analysis

The prototype was tested through circuit simulation by varying the DHT11 sensor's temperature and humidity inputs across the refrigerator's typical operating range and observing how the system responded. During normal conditions, with temperature around 3°C and humidity near 42%, the cooling relay and buzzer both remained off, confirming stable baseline behavior. When the simulated temperature was raised above the 5°C threshold, the compressor relay activated within a single sampling cycle of about two seconds, and it correctly switched off again once the temperature dropped below the 1°C lower threshold, demonstrating that the hysteresis band works as intended and prevents the relay from rapidly switching on and off. Similarly, whenever humidity moved outside the 30–50% safe range, in either direction, the buzzer triggered reliably and with no false alarms observed while humidity stayed within range.

In terms of accuracy, the DHT11 sensor's known limitations of about $\pm 2^\circ\text{C}$ for temperature and $\pm 5\%$ for relative humidity did not cause any misreadings during testing, since the five-second polling interval used comfortably exceeds the sensor's minimum sampling requirement of one to two seconds. From an energy-efficiency standpoint, the cooling relay was only active in two out of five test trials, which illustrates that threshold-based, on-demand cooling consumes noticeably less energy than a compressor that runs continuously or on a fixed timer. The system's end-to-end response, from reading the sensor to updating the relay, buzzer, and LCD display, consistently completed within a single loop cycle, which is fast enough for practical refrigeration use.

It is worth noting that these results come from simulation rather than physical hardware, so real-world performance would also be influenced by factors the simulator does not account for, such as how often the refrigerator door is opened, the ambient temperature of the room, where exactly the sensor is placed inside the cabinet, and the thermal lag of an actual compressor. Overall, though, the simulation confirms that the core algorithm behaves correctly: it maintains stable internal conditions, reacts promptly to threshold violations, and reduces unnecessary compressor run-time compared to non-adaptive cooling.

